

DD2352 Algorithms and Complexity
Spring 2026
Lecture 11: P, NP, and NP-completeness

Per Austrin

2026-03-24

Agenda

Recap and Reductions

The Complexity Classes P and NP

NP-completeness

NP-completeness examples

Types of Problems (Recap)

Three main types of problems:

Decision (yes/no)

Optimization (find maximum/minimum value)

Search (find solution object)

In this part of the course, main focus will be on **decision problems**.

In general we do not lose much by this, optimization and search for the problems we work with can be reduced to (= easily solved using an algorithm for) to the decision version.

(See exercise class this week!)

Turing Reductions (Recap)

A **Turing reduction** from a problem X to a problem Y is an efficient algorithm for problem X which is allowed to assume an algorithm for Y as a black box.

```
1 function SolveX(A):  
2   Do some stuff  
3   Build some instance  $B$  of problem  $Y$   
4   Call SolveY( $B$ ) and use the answer somehow  
5   Do other stuff  
6   Call SolveY( $B'$ ) on some other instance  $B'$   
7   Do more stuff  
8   ...  
9   ???  
10  ...  
11  Figure out answer to the input  $A$ 
```

Notation: we write $X \leq_T Y$ to denote this (“ X Turing-reduces to Y ”)

Example Reductions (Recap)

In last lecture, we looked at a few reductions and established

$$3\text{-SAT} \leq_T \text{INDEPENDENT SET} \leq_T \text{VERTEX COVER} \leq_T \text{SET COVER}$$

This shows that these problems are at least as hard as 3-SAT is.

Karp Reductions

All the reductions we saw yesterday were like this:

```
1 function SolveX(instance A of problem X):  
2   Create an instance B of problem Y in some way from A in  
   polynomial time  
3   return SolveY(B)
```

A reduction of this form is called a **Karp Reduction** (or a *polynomial-time many-one reduction*).

Notation: we write $X \leq_p Y$ to denote this (“*X* Karp-reduces to *Y*”)

In words: $X \leq_p Y$ if given any instance *A* of problem *X*, we can in polynomial time construct an instance $f(A)$ of problem *Y* such that “answer for *A*” = “answer for $f(A)$ ”.

Karp Reductions vs Turing Reductions

Karp reductions are a specialized form of Turing reductions.

If $X \leq_p Y$ then $X \leq_T Y$.

But it could be that $X \leq_T Y$ and $X \not\leq_p Y$.

Caveat: *the book never introduces Karp reductions and only defines Turing reductions. This leads to subtle but crucial differences in the material about NP-completeness that follows. We will follow the standard definitions and use Karp reductions.*

Lab 2

$X \leq_p Y$ if there is a reduction of this form:

```
1 function SolveX(instance A of problem X):  
2   Create an instance B of problem Y in some way from A in  
   polynomial time  
3   return SolveY(B)
```

Lab 2 in the course is about creating a reduction like this.

In particular your job is to implement “line 2” above.

- ▶ *Theory part*: you should be ready to start this week.
- ▶ *Programming part*: mostly ready to start after today (but Lecture 12 may give additional inspiration).

(Warning: you will probably initially find it confusing what exactly X and Y should be in the lab. But figuring out what X and Y are, and *why* they are what they are, is an important part of understanding the lab.)

Agenda

Recap and Reductions

The Complexity Classes P and NP

NP-completeness

NP-completeness examples

Efficient Algorithms

The basic convention in computational complexity is

Efficient = Polynomial Time

This “definition” is not an objective mathematical truth but a pragmatic choice, motivated by both theory and practice.

By this standard, a complexity of $\Theta(n^{10000})$, or an algorithm doing $10^{100} n \log n$ operations is efficient, and $O(1.0001^n)$ time is inefficient, even though the latter is probably preferable in practice.

But in practice such “pathological” cases (mostly) do not appear.

- ▶ Most polynomial time algorithms have reasonably small exponent and constant factor e.g. $2n^3$ operations.
- ▶ Most exponential time algorithms have time complexity like $O(2^n)$ rather than $O(1.0001^n)$.

Let us now give a formal definition based on this convention.

Decision Problems as Languages

A **formal language** over $\{0, 1\}$ is a set of binary strings

A decision problem P can be identified by a formal language L_P defined by $x \in L_P$ if and only if x is a “yes”-instance of P

In other words, we can think of a decision problem as simply being a set of inputs (those where the answer is “yes”).

Decision Problems as Languages, Example

VERTEX COVER

Input: Graph G and parameter k

Output: Does G have a vertex cover of size $\leq k$?

How is this a set of binary strings?

First, let us agree on some text representation of instances.

For example: first one line containing $|V|$, $|E|$, and k , then $|E|$ lines each containing two integers describing an edge.

4 5 2

1 2

2 3

3 4

4 1

2 4

Decision Problems as Languages, Example

VERTEX COVER

Input: Graph G and parameter k

Output: Does G have a vertex cover of size $\leq k$?

How is this a set of binary strings?

Second, encode the text as binary in some reasonable way.

For example: take the ASCII encoding of the text.

```
00110100001000000011010100100000
00110010000010100011000100100000
00110010000010100011001000100000
00110011000010100011001100100000
00110100000010100011010000100000
00110001000010100011001000100000
0011010000001010
```

Vertex Cover as a Language

We can encode an instance as a binary string, e.g.:

001110010010000000110000001000000011001100001010

The Vertex Cover “language” is then the set of all binary strings that are proper encodings of “yes” instances.

Note! Exactly what set of strings this language is depends on how we chose to encode instances.

- ▶ We could have chosen a different text representation
- ▶ We could have chosen a different way to encode text as binary
- ▶ We could have encoded the graph in binary in some direct way instead of going via a text format

However, these details are mostly irrelevant. *As long as we can convert between two different encodings in polynomial time, it does not matter which of them we choose.*

P

P is the set of all decision problems that can be solved in Polynomial time.

More formally:

A language L is in **P** if there exists a constant k and algorithm $A : \{0, 1\}^* \rightarrow \{\text{yes}, \text{no}\}$ with time complexity $O(n^k)$ for deciding L .

(Saying that A *decides* L means that $A(x)$ answers whether $x \in L$.)

In other words, **P** consists of the set of problems that have efficient algorithms (under our choice of “efficient = polynomial time”)

But polynomial time in which model of computation?

The Church-Turing Thesis

Recall the Church-Turing Thesis (CT):

Any realistic model of computation can be simulated by a Turing machine.

The CT says nothing about time complexity.

Perhaps a Turing machine would require much more time?

Maybe some problem can be solved in polynomial time in some other model of computation (like the RAM) although it needs exponential time on a Turing machine?

The Extended Church-Turing Thesis

The **Extended Church-Turing Thesis** (ECT):

Any realistic model of computation can be efficiently simulated by a Turing machine.

(“efficiently” = “with a polynomial time overhead”)

The ECT is *probably false*!!

Probable counter-example: Quantum computing

However, the ECT *is* true for the classic models of computation underlying our laptops/phones/... (RAM model etc)

Bottom line: it does not matter if we define **P** as set of problems that can be solved in poly-time on Turing Machine or in RAM model, because these are the same.

Efficient verification

Last lecture we established

$$3\text{-SAT} \leq_p \text{INDEPENDENT SET} \leq_p \text{VERTEX COVER} \leq_p \text{SET COVER}$$

Not clear which of these problems can be efficiently solved, but they all share one interesting property: **efficient verification**

Concretely: when answer is “yes”, there is a simple **certificate** that can be provided and which can easily be used to verify that the answer is indeed “yes”.

For example let us look at 3-SAT.

3-SAT (reminder)

Input: $\Phi = (x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_2 \vee \neg x_{17}) \wedge \dots$

Output: is there an assignment to the variables that satisfies Φ ?

Efficient verification for 3-Sat

Given 3-SAT instance Φ on n variables $x_1, \dots, x_n$, one possible certificate that Φ is satisfiable is an assignment to the variables.

```
1 function Check3Sat( $\Phi, \alpha$ ):  
2   if  $\alpha$  is not a list of  $n$  Boolean values then return No  
3   foreach clause of  $\Phi$  :           /* (e.g.  $(x_i \vee \neg x_j \vee x_k)$ ) */  
4     if  $\alpha$  does not satisfy the clause then  
5       // (e.g. if  $\alpha[i] = \alpha[k] = \text{False}$  and  $\alpha[j] = \text{True}$ )  
6       return No  
7   return Yes
```

If Φ has n variables and m clauses, algorithm can easily be implemented in time $O(m + n)$.

Note:

Φ satisfiable $\Rightarrow$ exists α such that Check3Sat(Φ, α) returns Yes.

Φ not satisfiable $\Rightarrow$ Check3Sat(Φ, α) returns No *for every* α .

NP

NP is the set of all decision problems that have polynomial-time verification.

In other words, in the cases when the answer is “yes”, there is a simple **witness**, or **certificate**, that can be provided to convince us that the answer is “yes”.

A bit more formally:

a language L is in **NP** if there exists a polynomial p and polynomial-time algorithm $A : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{\text{yes}, \text{no}\}$ such that

- ▶ if $x \in L$ then there is a y of length $|y| \leq p(|x|)$ such that $A(x, y) = \text{yes}$
- ▶ if $x \notin L$ then $A(x, y) = \text{no}$ for all y

NP stands for **N**ondeterministic **P**olynomial time

(Please do not ever think it stands for “Non-Polynomial”!)

Different views of NP

Two other ways of viewing NP:

- ▶ NP as **nondeterministic** computation (the historic origin)
a nondeterministic computation is one where we can make nondeterministic guesses, and we always make the right guess to reach a “yes” answer
- ▶ NP-witnesses as “proofs”:
view an instance of an NP problem as a mathematical claim
a witness is a proof of that claim

Some first facts about NP

The four problems we studied earlier are all in **NP**

INDEPENDENT SET $\in$ **NP**

VERTEX COVER $\in$ **NP**

SET COVER $\in$ **NP**

3-SAT $\in$ **NP**

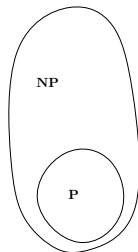
Membership in **NP** is maintained under Karp reductions:

If $Y \in \mathbf{NP}$ and $X \leq_p Y$ then $X \in \mathbf{NP}$.

Not true for Turing reductions!

All problems in **P** are also in **NP** (do you see why?)

$$\mathbf{P} \subseteq \mathbf{NP}$$



The P vs NP Question

$P \subseteq NP$, but what about the converse?

Are all problems in NP also in P ?

The greatest open question about computation: “Is $P = NP$?”

Informally: “Can mathematical creativity be automated?”

We do not know (and we are not going to find out any time soon).

The majority belief is very strongly that $P \neq NP$
(but there is not much hard evidence for this)

Agenda

Recap and Reductions

The Complexity Classes P and NP

NP-completeness

NP-completeness examples

NP-hardness

We say that a problem Y is **NP-hard** if *every problem in NP* can be Karp-reduced to Y

I.e., if $X \leq_p Y$ for every $X \in \mathbf{NP}$

Intuitive meaning: Y is at least as hard as every problem in **NP**

If Y has an efficient algorithm then *all* problems in **NP** do as well

Conversely, if there exists *some* problem in **NP** which is difficult to solve (has no poly-time algorithm), then so is Y .

NP-completeness

Definition

A problem Y is **NP-complete** if

- (i) $Y \in \mathbf{NP}$, and
- (ii) Y is **NP-hard**.

Intuition:

the **NP**-complete problems are the hardest problems in **NP**
(being **NP**-hard, they are at least as hard as every problem in **NP**)

NP-completeness, What's the Big Deal?

At first glance, it is far from obvious whether this is a useful definition – not even clear whether **NP**-complete problems exist.

Even if they do exist, seems like a pretty unwieldy notion to work with: to prove **NP**-hardness for Y we need to prove that *all* problems in **NP** can be reduced to Y .

Or do we?

Nice feature of **NP**-hardness: if X is **NP**-hard and $X \leq_p Y$, then Y is **NP**-hard as well.

In other words: to prove **NP**-hardness for Y we *only have to find one other NP-hard problem X and reduce X to Y* , we don't have to reduce all problems in **NP** to Y .

The CNF-Sat Problem

A CNF formula is a logic formula of the form:

$$(x_1 \vee x_2) \wedge (x_4 \vee \neg x_2 \vee \neg x_{17} \vee x_{4711}) \wedge \dots$$

In words:

- ▶ A **conjunction** (logical AND) of a bunch of **clauses**
- ▶ Each clause is a **disjunction** (logical OR) of some number of **literals**
- ▶ Each literal is either a variable or a negated variable.

(CNF = Conjunctive Normal Form)

SAT

Input: A CNF formula Φ .

Output: Is Φ satisfiable?

Previously we talked about the closely related 3-SAT problem. Only difference is that in 3-SAT, all clauses have exactly 3 literals.

The Cook-Levin Theorem

Theorem (Cook-Levin, ≈ 1971)

The SAT problem is NP-hard.

This is a remarkable result!

Next lecture we will see how such a theorem can be proved.

The SAT problem is also in **NP**
(given a satisfying assignment we can check it in polynomial time)

We have our first **NP**-complete problem!

Agenda

Recap and Reductions

The Complexity Classes P and NP

NP-completeness

NP-completeness examples

3-Sat

If SAT is **NP**-hard, perhaps also 3-SAT is?

We have that $3\text{-SAT} \leq_p \text{SAT}$
(because 3-SAT is a special case of SAT)

Does it follow that 3-SAT is **NP**-hard?

No! That's the wrong direction of the reduction!

To establish **NP**-hardness of 3-SAT, we would need to show
 $\text{SAT} \leq_p 3\text{-SAT}$

Reduction from Sat to 3-Sat

Given a SAT instance Φ , we construct a new 3-SAT instance Ψ as follows.

- ▶ Clauses of length ≤ 3 in Φ we keep as they are (since they are allowed in 3-SAT)
- ▶ For a clause $(x_1 \vee x_2 \vee \dots \vee x_k)$ of length $k > 3$ in Φ , we introduce $k - 3$ new variables $y_1, y_2, \dots, y_{k-3}$, and create the following $k - 2$ new clauses of size 3:

$$(x_1 \vee x_2 \vee \neg y_1)$$

$$(y_1 \vee x_3 \vee \neg y_2)$$

$$(y_2 \vee x_4 \vee \neg y_3)$$

...

$$(y_{k-4} \vee x_{k-2} \vee \neg y_{k-3})$$

$$(y_{k-3} \vee x_{k-1} \vee x_k)$$

- ▶ The intuition behind the y variables is that y_i is an indicator of whether at least one of the first $i + 1$ x -variables are true.

Reduction from Sat to 3-Sat, example

For the input formula

$$\begin{aligned}\Phi = & (x_2 \vee x_7 \vee \neg x_3 \vee x_1 \vee \neg x_5) \wedge \\ & (x_3 \vee x_1 \vee x_2) \wedge \\ & (\neg x_1 \vee \neg x_2 \vee \neg x_5 \vee \neg x_9)\end{aligned}$$

The reduction produces the formula

$$\begin{aligned}\Psi = & (x_2 \vee x_7 \vee \neg y_1) \wedge (y_1 \vee x_3 \vee \neg y_2) \wedge (y_2 \vee x_1 \vee \neg x_5) \wedge \\ & (x_3 \vee x_1 \vee x_2) \wedge \\ & (\neg x_1 \vee \neg x_2 \vee \neg z_1) \wedge (z_1 \vee \neg x_5 \vee \neg x_9)\end{aligned}$$

Note:

For each clause of Φ , some completely new variables are created.
E.g. y_1 and y_2 from the first part are not reused anywhere else.

Analyzing the reduction

To prove that a reduction like this is correct, we need to establish three things:

1. The reduction is polynomial-time

This is often very easy or even obvious.

2. Reduction maps “yes” instances into “yes” instances.

If original SAT instance Φ is satisfiable, then new 3-SAT instance Ψ produced by the reduction is also satisfiable.

3. Reduction maps “no” instances into “no” instances.

If new 3-SAT instance Ψ is satisfiable, then original SAT instance Φ is also satisfiable.

This is often the most difficult part.

Proofs of these properties for the reduction from SAT to 3-SAT sketched during lecture.

Examples from Previous Lecture

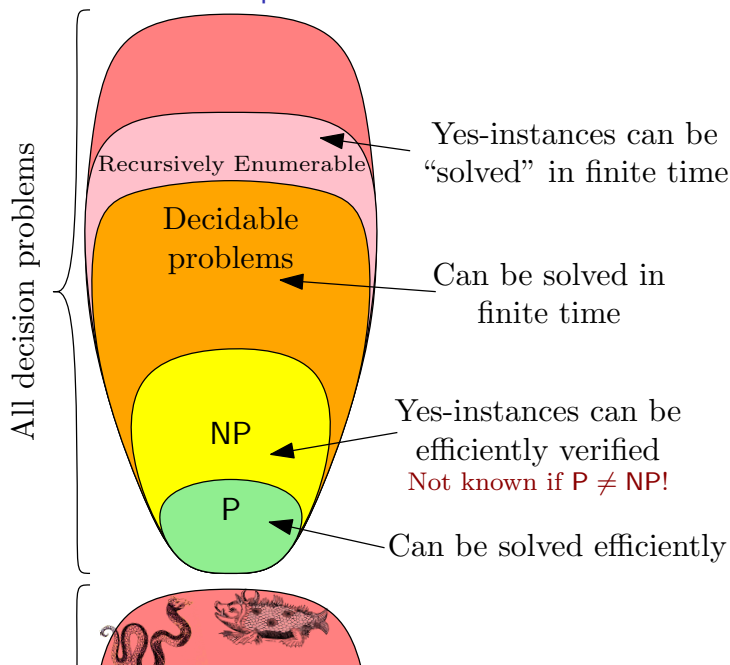
In last lecture we saw the following chain of relations:

$$3\text{-SAT} \leq_p \text{INDEPENDENT SET} \leq_p \text{VERTEX COVER} \leq_p \text{SET COVER}$$

We just saw that 3-SAT is **NP**-hard, and from this it follows that the other three problems are **NP**-hard as well.

As observed earlier all four problems are in **NP**, so they are all **NP**-complete.

Overview of landscape so far



Upcoming Week In The Course

- ▶ Thu 15-17: Exercise 7
Undecidability and reductions
(Check out the problems about decision vs optimization vs search even if you usually skip the exercises!)
- ▶ Friday 8-10: first lab session for Lab 2
Drop-in only
Primarily for help, but can present if you are done Can also present Lab 1
- ▶ Tue 13-15: Lecture 12
More **NP**-completeness examples and the Cook-Levin Theorem